B.TECH RESEARCH PROJECT · SVNIT SURAT

Smart Home Energy Planner — Surat

Forecast-aware hybrid agentic AI for Building-to-Grid demand response

What this document is: a complete, plain-language explanation of our project for the team: the idea, the background you need, every step we built, the results, and how to run and present the demo. No prior knowledge of energy systems is assumed. Read it in order; each part builds on the previous one.

The project in one minute

A house in Surat has rooftop solar, a home battery, an electric car and an air conditioner. The person types a request in plain English, e.g. *"I'm leaving at 7 tomorrow, I need the car at 90%, keep the room under 26 °C."*

Our system plans the whole next day in 15-minute steps: when to charge the car, when to charge or use the battery, and when to run the AC, so that the family pays as little as possible, uses its own solar, stays comfortable and never overloads the 8 kW grid connection.

The key design: the **LLM understands** the request, an **optimizer does the maths**, and a **verifier checks** the result (and sends it back for replanning if something is wrong).

Main results

33–46% less electricity than a normal "simple rules" house on our four example days.

Within about 5% of a planner that knew the real weather in advance (the best possible).

More comfortable than simple rules on every example, and always under the 8 kW grid limit.

A pure LLM planner failed badly: it broke 51 physical rules, left the car 20 kWh short and let the room reach 30 °C.

The verifier **caught the AI's own mistakes** (an invented temperature setting) and the agents fixed them.

Contents

1. Electricity basics
2. The problem: timing
3. The house and its devices
4. The base paper we started from
5. Our idea: understand, compute, check
6. The data
7. Forecasting
8. The physics of the house
9. The optimizer (MILP) and how we score plans
10. The AI agents
11. The verifier and replanning
12. Results
13. The demo
14. Limitations and future work
15. Questions we may be asked
16. Glossary
17. Appendix: milestones, code map and commands

PART 1 · BACKGROUND

Electricity basics

Five ideas are enough to understand everything else in this document.

1. Power (kW) and energy (kWh)

Think of electricity like water from a tap. **Power (kW)** is how *fast* it flows right now. **Energy (kWh)** is how *much* you collected in total. **Energy = power × time.** Electricity bills charge for energy; in India 1 kWh is "1 unit" on the bill.

Example	Power	Time	Energy
Car charger	7.4 kW	2 hours	14.8 kWh
Air conditioner	1.6 kW	5 hours	8 kWh
Ceiling fan	0.075 kW	10 hours	0.75 kWh

2. The grid

The grid is the power company's network. Taking electricity from it is **import** (you pay). Sending your extra electricity to it is **export** (you get a small credit, much less than the import price). So it is better to **use your own solar yourself** than to export it and buy power back later.

3. Solar panels

Panels produce only while the sun shines: zero at night, highest around noon. Our 5 kW system produces about **21.9 kWh on an average day** in Surat, less on cloudy days, and much less during the monsoon (June–September).

4. Batteries

A battery is like a water tank: fill it when there is extra, use it later. Our home battery stores **10 kWh** and can charge or discharge at up to **5 kW**. A little energy is lost each time (we assume 5% going in and 5% coming out).

5. The price changes during the day

We use a **time-of-day tariff**: cheap when the sun shines, expensive in the evening when everybody uses power. The numbers are placeholders shaped like Indian time-of-day tariffs, not a real bill.

Time	Price per kWh	Why
00:00 – 09:00	₹7.0	normal
09:00 – 17:00	₹5.6 (cheapest)	solar hours, plenty of power
17:00 – 18:00	₹7.0	normal
18:00 – 22:00	₹8.4 (most expensive)	evening peak, the grid is stressed
22:00 – 24:00	₹7.0	normal

PART 2 · BACKGROUND

The problem: timing

Over a whole day there is enough solar: on average the panels make **21.9 kWh** while the house's basic needs (lights, fridge, fans, cooking) use about **10.6 kWh**. The problem is **when** each happens.

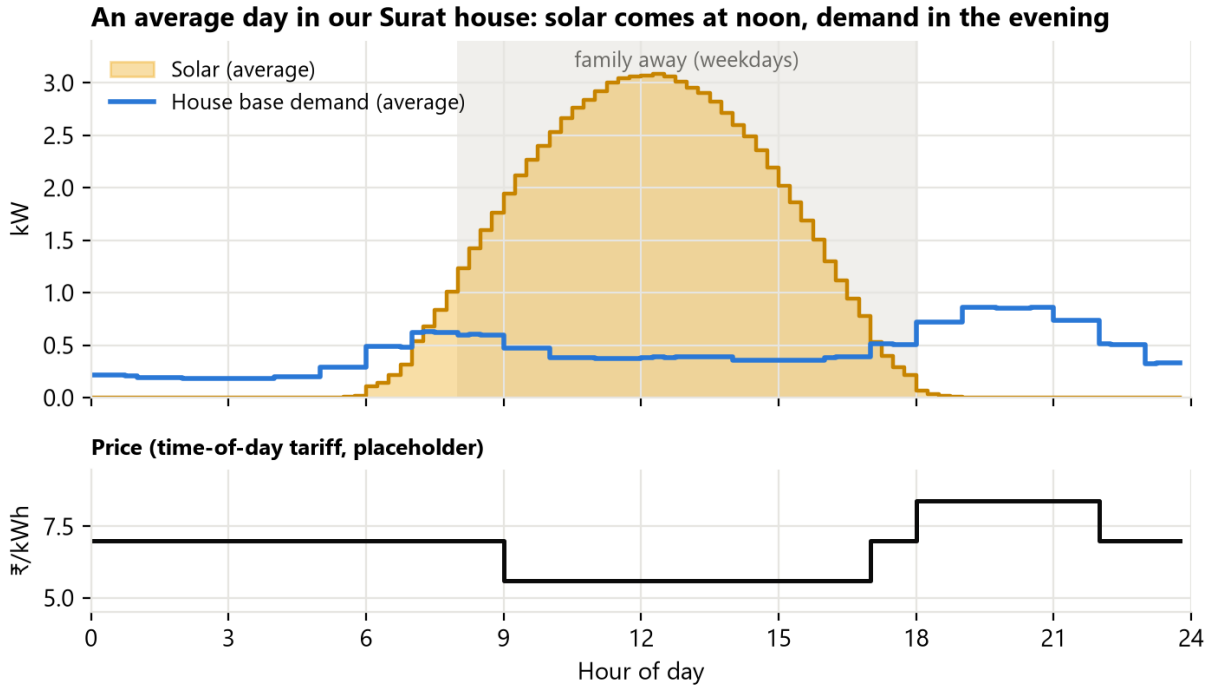


Figure 1. Average of all 973 days in our data. Solar (yellow) peaks at noon while the family is away; house demand (blue) peaks in the evening, exactly when the price is highest.

The big devices make it worse

A normal household plugs in the **car (7.4 kW)** and switches on the **AC (1.6 kW)** when it comes home at 18:00: peak price, zero solar. Together with the other appliances that is about 9.5 kW, which is **more than the 8 kW the grid connection allows**. So the devices are not independent; they must be planned **together**.

Tomorrow is uncertain

A plan for tomorrow can only use a **forecast** of tomorrow's sun. The day-before forecast of sunlight is off by about **9.6%** on average, and much more on monsoon days. The system must plan with forecasts and still behave sensibly when they are wrong.

The fix: shift flexible loads

Some electricity use cannot move (lights when it is dark). Some **can** move without anyone noticing. These **flexible loads** are the heart of our project:

Flexible load	Why it can move	Smart timing
Car charging	It only has to be full *by* departure	Charge at night, spread out, under the 8 kW limit
Home battery	It is storage	Fill it with free midday solar, use it in the evening peak
Pre-cooling (AC)	A house warms up slowly, like a cold bottle in a bag	Cool the house in the sunny, cheap afternoon so it is still cool in the evening

Where the course topic comes in

Demand response means changing *when* you use electricity to help the grid: less at peak times, more when there is plenty.

Building-to-Grid (B2G) means the building behaves as a good partner of the grid: it uses its own solar, avoids the evening peak and keeps its draw low and steady. A **microgrid** is a small local network with its own generation (a house, a campus, a village). If thousands of homes do this, the grid needs fewer expensive power plants for the evening peak. That is why this topic matters beyond one family's bill.

PART 3 · SETUP

The house and its devices

All numbers below are set in one settings file, `b2g/config/default.yaml`. They are realistic **placeholders** for a middle-class Indian home with solar and an EV, not measurements of a real house. Changing the file changes the whole project without touching any code.

Time is cut into 96 slots

The system does not think in hours: each day is cut into **96 slots of 15 minutes** (slot 0 = 00:00–00:15, slot 28 = 07:00–07:15, slot 95 = 23:45–24:00). A plan is a table with 96 rows saying what each device does in each slot.

The devices

Device	Settings	Controllable?
Solar panels	5 kW peak, 20% losses. Power = 5 kW × (sunlight ÷ 1000 W/m^2) × 0.8. About 21.9 kWh/day.	No (follows the sun)
Base load	Lights, fridge, fans, cooking: about 10 kWh on weekdays, 12 kWh on weekends; highest in the evening. Synthetic (pattern + noise).	No
Home battery	10 kWh; up to 5 kW charge or discharge; 95% efficiency each way; never below 10%; starts the day at 50%.	**Yes**
Electric car	40 kWh battery, 7.4 kW home charger, 92% charging efficiency. Weekdays: leaves 08:00 needing 80% (32 kWh), back 18:00; the trip uses 16 kWh. Weekends: stays home. Can charge only while at home.	**Yes**
Air conditioner	1.5-ton split AC: 1.6 kW of electricity removes about 5.3 kW of heat (COP 3.3). Comfort band 24–27 °C, only while someone is home (weekdays 18:00–08:00, weekends all day).	**Yes** (pre-cooling)

Device	Settings	Controllable?
Grid connection	Import limit 8 kW at any moment; export credit ₹2.5/kWh; time-of-day price from Part 1.	Result of the others

The balance rule in every slot

Whatever the plan does, in every 15-minute slot energy in must equal energy out:

> **solar + battery discharge + grid import = base load + AC + car charging + battery charging + export**
>
> The grid fills whatever is missing, but never more than 8 kW.

The usual routine and how a request changes it

The house has a default routine (the "calendar"): the car leaves at 08:00 needing 80% and returns at 18:00, nobody is home 08:00–18:00 on weekdays, comfort is 24–27 °C, the battery reserve is 10%. A request **edits only parts** of it:

The request says	What changes
"leaving at 7 instead of 8"	car departure 08:00 → 07:00
"need the car at 90%"	target 80% → 90% (36 kWh)
"working from home from 2 pm"	someone home again from 14:00 instead of 18:00
"keep it under 26 °C"	comfort maximum 27 → 26 °C
"keep 30% in the battery for power cuts"	battery reserve 10% → 30% (3 kWh)

Turning the sentence into these exact edits is the job of the LLM agents (Part 10).

PART 4 · BACKGROUND

The base paper we started from

We extend a published paper and its open-source code (MIT licence, so we may reuse and modify it):

> El Makroum, Zwickl-Bernhard, Kranzl (2025). *Agentic AI Home Energy Management System: A Large Language Model Framework for Residential Load Scheduling.* arXiv:2510.26603. Code: github.com/RedaElMakroum/agentic-ai-hems

Two words first

- **LLM (large language model):** an AI that reads and writes text, like ChatGPT. Very good at understanding language; not reliable at exact arithmetic over many numbers.
- **Agent:** an LLM given a specific job and some tools it may use (for example "get prices"). Instead of only chatting, it takes actions. A **multi-agent system** has several specialist agents and a manager agent, the **orchestrator**.

What the paper built

A home energy manager for a house in Austria. An orchestrator LLM receives a request such as "schedule all flexible loads for minimal cost" and asks specialist agents for the washing machine (2.0 kW, 120 min), dishwasher (1.8 kW, 90 min) and EV charger (7.4 kW, 360 min) to choose start times, using real Austrian market prices in 96 slots. Each appliance is simply on or off for a fixed duration.

The orchestrator works in a **ReAct** loop (**Re**ason + **Act**): think, take an action, observe the result, think again. Actions are written as one line that the program executes:

```
Thought: I need today's prices first.
ACTION: GET_PRICES
Observation: [96 prices ...] cheapest around 13:00-15:00
Thought: Now ask the dishwasher agent for the best start time.
ACTION: CALL_AGENT | agent=dishwasher
Observation: dishwasher -> start 13:15
ACTION: FINISH | summary=Dishwasher at 13:15, ...
```

Its weaknesses: our opportunity

#	Weakness	Why it matters
1	The LLM does the scheduling arithmetic itself	LLMs make number mistakes; no guarantee the plan is correct or even possible
2	No solar or weather forecast, only prices	It cannot use free solar, so the core B2G idea is missing
3	No battery, no AC / pre-cooling	The two most useful flexible loads for an Indian home are absent
4	Appliances planned separately	Nothing makes them fit together under a grid limit
5	Nothing checks the result	Mistakes, and wrong explanations, go unnoticed
6	Practical issues	Hard-wired to one paid AI provider; crashed without an Austrian price API key

What we kept and what we changed

We kept the agent idea and the ReAct style with `ACTION: NAME | key=value`, the 96 slots and the 7.4 kW charger. We also kept **the original system itself runnable** as a baseline (called **S0**); we only fixed what blocked it (any AI provider can be used, the price key is optional). All our new code is in the `b2g/` folder.

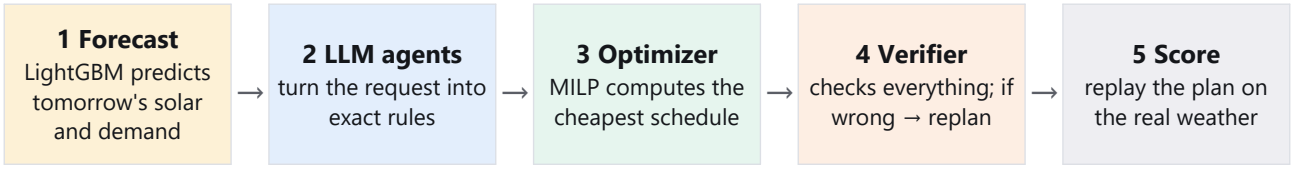
Base paper	Our project
Prices only	Solar and weather forecasts for Surat (real data)
Washing machine, dishwasher, EV	EV + home battery + AC pre-cooling
Each appliance on its own	All devices planned together under the 8 kW limit
The LLM picks the times	The LLM sets the rules; a MILP optimizer computes the schedule
No checking	A verifier checks physics, wishes and the explanation, and triggers replanning
Judged on the given prices	Planned on the forecast, scored on the real weather

Base paper	Our project
Austria	Surat, India (Indian-style tariff, monsoon, ₹)

PART 5 · THE CONTRIBUTION

Our idea: understand, compute, check

Our contribution in one line: **a forecast-aware, hybrid agentic system: LLM agents understand the request and set goals and constraints, a MILP solver computes the schedule, and a verifier checks physical limits and triggers replanning, coordinating EV, battery and pre-cooling under shared solar and a grid import limit.**



Why split the work this way?

Each part does the job it is good at, and only that job:

Part	Good at	Bad at, so it does not do it
LLM agents	Understanding messy human language (“half an hour earlier”, “keep it cool”, “full”)	Exact arithmetic over 96 slots × several devices
Optimizer (MILP)	Finding the mathematically best schedule that obeys every rule	Understanding sentences
Verifier (plain code)	Checking facts reliably, the same way every time	Being creative; it only checks
Forecaster (ML)	Predicting tomorrow's solar and demand from weather forecasts	Knowing about the person's plans

“Hybrid” means we combine an AI that reasons in language (the LLM) with classical, exact methods (optimization and rule checks). “Agentic” means the LLM works as agents that take actions with tools. “Forecast-aware” means the plan uses predictions of tomorrow's weather.

> **An important rule of our evaluation**
>
> The plan is made the day before using **forecasts**. To score it fairly we replay it on the **actual** weather of that day (Part 9). We never grade a plan against the same forecast it was made with: that would hide forecast errors and make every system look better than it is.

How we built it: the milestones

Milestone	What we built	Part
M0	Ran and reproduced the original paper's system (our baseline S0)	4
M1	One shared LLM client: any provider, logging, caching, retries	17
M2	Real Surat weather and solar data, 15-minute steps	6

Milestone	What we built	Part
M3	Forecasters for solar and house demand	7
M4	Physical models: battery, car, room temperature, power balance	8
M5	The MILP optimizer, the settlement on real weather, comparison systems	9
M6	The LLM agents (orchestrator + EV, battery, pre-cooling specialists)	10
M7	The verifier and replanning; the pure-LLM comparison (S3)	11
M8	The demo (website with live requests)	13

PART 6 · MILESTONE M2

The data

We need two versions of the weather for every day: what was **forecast** the day before (what a planner would know) and what **actually** happened (what the plan is scored on).

What	Source	Notes
Actual weather	Open-Meteo archive (ERA5 reanalysis)	A scientific reconstruction of the real weather, hour by hour
Forecast weather	Open-Meteo "previous runs" API	The weather model's forecast issued about one day ahead
Variables	Sunlight (W/m^2), temperature (°C), cloud cover (%)	For Surat: 21.17° N, 72.83° E
Period	1 Feb 2024 – 30 Sep 2026	**973 days**, three monsoons
Solar power	Computed from sunlight	5 kW × sunlight/1000 × 0.8, for both actual and forecast
House base load	Synthetic	Realistic daily shape + seeded random noise (no smart meter available)
Price	Time-of-day tariff	From Part 1 (placeholder)

Hourly weather is converted to 15-minute slots in a way that keeps the total energy the same. The result is one table (`data/b2g/processed/site_15min.csv`) with one row per 15 minutes and columns for actual and forecast sunlight, temperature and cloud, solar power, base load and price.

Numbers worth remembering

- Average sunlight: **5.47 kWh/m^2 per day**; our panels: **21.9 kWh/day** on average.
- House base load: about **10.6 kWh/day**.
- Day-ahead sunlight forecast error: about **9.6%** on average (the forecast tends to be slightly too low).

PART 7 · MILESTONE M3

Forecasting

Before planning tomorrow, the system predicts tomorrow's solar power and house demand for each 15-minute slot. We compared simple methods with a machine-learning model:

Method	Idea
Persistence	Tomorrow = today (the simplest possible guess)
Same day last week	For demand: next Monday looks like last Monday
Weather physics	Put the weather forecast's sunlight into the solar formula
LightGBM	A machine-learning model (many small decision trees) that learns from the weather forecast, hour, weekday and season

Honest testing: split by time

A forecaster must be tested on days it has never seen. We split the data **by time**, never randomly:

- **Training** (the model learns): up to 30 June 2025.
- **Validation** (we choose the model's settings): July – December 2025.
- **Test** (final score only, untouched until the end): January – September 2026. All our example days are in this period.

Results on the test period (error as % of the average value, called nMAE; lower is better):

	Persistence	Weather physics	Same day last week	LightGBM
Solar	9.8%	**7.0%**	–	8.1%
House demand	20.6%	–	16.8%	**12.0%**

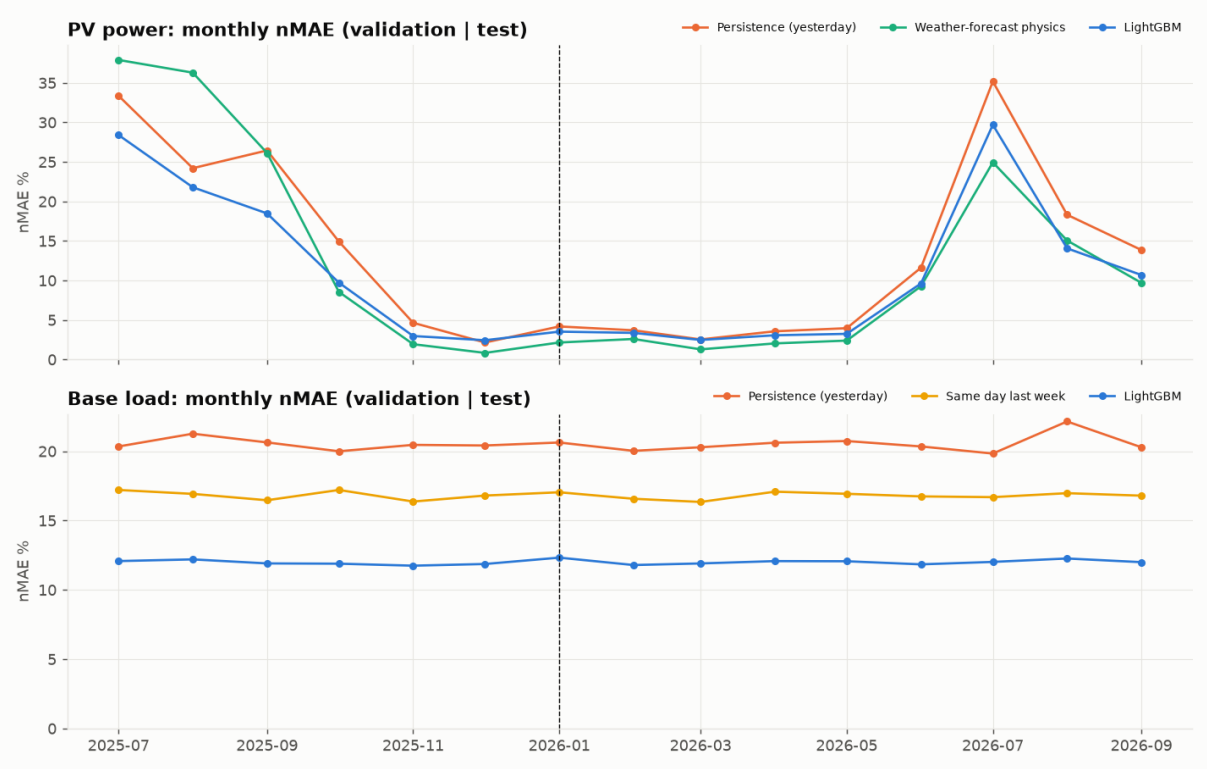


Figure 2. Forecast error by month. The monsoon months are the hardest for solar.

What we learned

- For **demand**, LightGBM is clearly best. (About 12% is close to the random noise we put into the synthetic demand, so it cannot do much better.)
- For **solar**, LightGBM won on validation (13.9% vs 18.5% for physics) but physics won on the test period. The weather model's own systematic error (its bias) changed in 2026, so what LightGBM had learned from the older years no longer helped.
- This is called **drift**: data changes over time, so a model chosen on old data is not always best on new data. It is a real, reportable finding. The planner uses the LightGBM forecast (the one chosen on validation, as a fair process requires).

PART 8 · MILESTONE M4

The physics of the house

To plan and to score plans we need small, tested models of how each device behaves. All work in 15-minute steps.

Model	What it does
Battery	Energy stored changes by charge × 95% minus discharge ÷ 95%. It cannot go above full, below 10%, or above 5 kW.
Car	Charges only when at home, at up to 7.4 kW with 92% efficiency; the trip removes 16 kWh; the target must be reached by departure.
Room temperature	A standard simple building model ("1R1C", one thermal resistance and one thermal capacity): the room drifts toward the outdoor temperature, warms from sunlight and people, and the AC removes heat. The house has thermal mass, like a heavy pot of water: it takes about a day to fully follow the outdoor temperature.
Site balance	Adds everything up in each slot (Part 3) and computes grid import and export, checking the 8 kW limit.

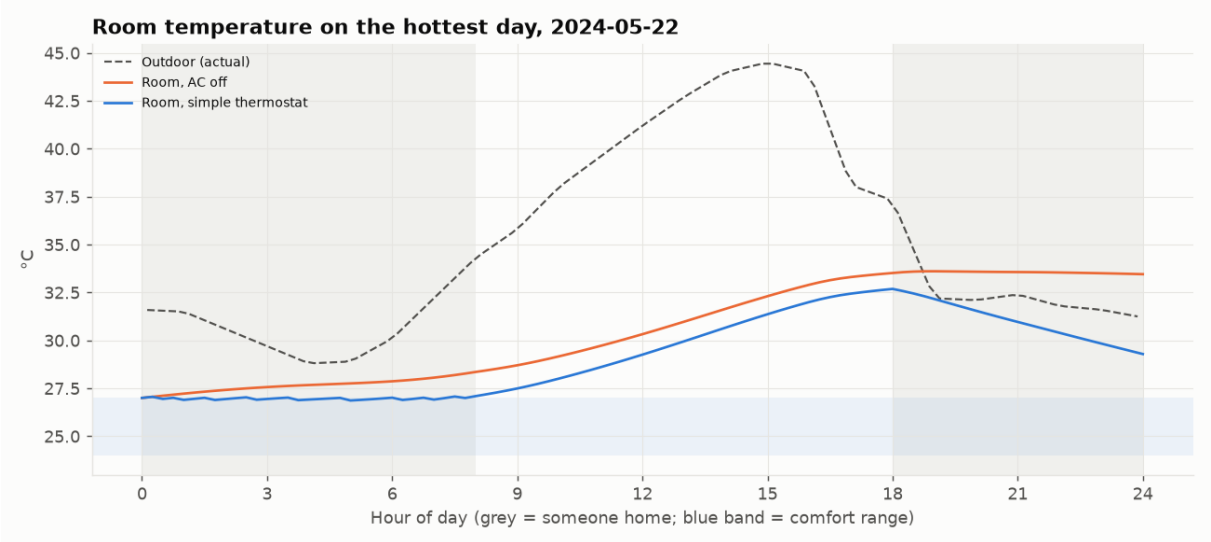


Figure 3. The hottest day in our data (22 May 2024, 44.5 °C outside). Grey = someone home, blue band = comfort range. A simple thermostat (blue line) only cools when people are home: the room heats up all day and at 18:00 it is 32.7 °C. The AC is too small to bring it back down quickly.

Finding

Waiting until people come home and then switching the AC on does not work on hot days: the AC is too small to catch up. Cooling the house **in advance** (pre-cooling, in the sunny and cheap afternoon) is what keeps people comfortable. This is why pre-cooling is one of our three flexible loads.

PART 9 · MILESTONE M5

The optimizer (MILP) and how we score plans

What is MILP?

Mixed-Integer Linear Programming is a mathematical method for finding the **best decision** that obeys a set of rules. You describe three things and a solver program finds the answer:

Ingredient	In our problem
Decisions	For each of the 96 slots: car charging power, battery charge and discharge, AC power, grid import and export
Rules (constraints)	The physics from Part 8; the balance rule in every slot; grid ≤ 8 kW; car at target by departure; room inside the comfort band while someone is home; battery within its limits
Goal (objective)	Minimise: electricity cost − export credit + battery wear (₹1.5 per kWh cycled) + comfort penalty (₹50 per °C·h outside the band) + car penalty (₹100 per kWh missing at departure) + a small peak penalty (₹1 per kW)

We use the free solver **CBC** through the Python library PuLP. One day solves in under a second on a laptop. Comfort and the car target are "soft": if something is truly impossible (e.g. a heat wave the AC cannot beat) the solver pays the penalty instead of failing, and the verifier later tells the person. "Mixed-integer" refers to an on/off variable that stops the battery from charging and discharging at the same time; we only add it when needed, which keeps solving fast.

Scoring on reality: the settlement

The plan is computed on the **forecast**. Then we **replay** it on the **actual** weather of that day. If there is less sun than forecast, the grid covers the difference and the bill goes up; if the battery cannot do what the plan said, its actions are clipped to what is physically possible. This is exactly what a real house would experience.

The systems we compare

System	What it is	Why it is there
S0 original	The base paper's system	Shows the baseline still works as published
S1 simple rules	Charge the car as soon as it is home, AC on a thermostat, battery stores spare solar	What a normal house does
S2 forecast + optimizer	The optimizer on the forecast, no LLM	Shows what forecasting + optimization alone achieve
S3 pure LLM	The LLM writes the whole 24-hour schedule itself	Tests whether an LLM alone can be trusted
S4 our system	Agents + optimizer + verifier	Our contribution
S5 perfect knowledge	The optimizer knowing the real weather in advance	The best possible score (cannot exist in reality)

One hot week without any LLM (4–10 May 2026), daily averages

System	Total ₹ (incl. penalties)	Electricity ₹	Discomfort °C·h	Peak kW	Solar used
S1 simple rules	628	153	8.93	6.40	67%
S2 (LightGBM forecast)	144	105	0.33	1.95	96%
S2 (physics forecast)	144	104	0.33	1.95	96%
S5 perfect knowledge	125	101	0.01	1.86	99%

Smart planning beats simple rules by a wide margin and, even with imperfect forecasts, comes close to perfect knowledge. ("°C·h" = degrees outside the comfort band × hours: 1 °C too warm for 2 hours = 2 °C·h.)

PART 10 · MILESTONE M6

The AI agents

The agents turn the person's sentence into exact rules for the optimizer. They never write the schedule themselves.

The orchestrator's actions

Action	What happens
GET_FORECAST	Read tomorrow's forecast: solar, outdoor temperature, demand
GET_CALENDAR	Read the usual routine (car times, who is home, comfort band)
CALL_AGENT	Ask a specialist to translate the part of the request that concerns it
RUN_SOLVER	Run the optimizer with the current rules and get a plan summary
VERIFY	Run the verifier (Part 11); fix and re-run if it fails
FINISH	Explain the plan to the person (only allowed after verification passes)

The three specialist agents

Agent	Can set
EV agent	departure time, arrival time, target charge, maximum charging power, skip charging
Battery agent	backup reserve, "charge only from solar"
Pre-cooling agent	comfort maximum and minimum, when the house is empty, "home all day"

Each agent answers in **JSON** (a strict machine-readable format) with **only the fields the request changes**. A schema (Pydantic) checks the format; if it is wrong the agent gets one chance to repair it. The LLM runs with **temperature 0** (no randomness), and every call is logged and cached (Part 17).

A full example

Request (Wednesday 6 May 2026, about 37 °C outside): *"Tomorrow I'm leaving at 7 instead of 8 and I need the car at 90%. I'm working from home from 2 pm, keep it under 26 °C. Keep at least 30% in the battery in case of a power cut."*

```
1  GET_FORECAST    solar, outdoor max about 37 C around 12:45, demand
2  GET_CALENDAR    usual: car 08:00-18:00 at 80%, nobody home 08:00-18:00
3  CALL_AGENT ev_agent       -> {"departure_time": "07:00", "target_soc": 0.9}
4  CALL_AGENT precool_agent  -> {"comfort_max_c": 26, "away_until": "14:00"}
5  CALL_AGENT battery_agent  -> {"reserve_soc": 0.3}
6  RUN_SOLVER      car 00:00-07:00 -> 36 kWh; battery charges from solar 09:45-15:45,
                   powers the evening; AC keeps the room as cool as it can
7  VERIFY          passed, 1 warning: room reaches 26.9 C around midday
8  FINISH          explanation to the person, including the 26.9 C warning
```

The whole run used 11 LLM calls (about 12,500 tokens) on Groq's free tier with the Qwen 3.8 27B model.

> **Finding**
>
> In early tests the LLM's final explanation sometimes **overclaimed**, e.g. "all your wishes are met" when the room actually reached 26.9 °C, or said the battery charged "from the grid" when it charged from solar. Language is the LLM's strength, but it still states things that are not true. This led directly to the explanation check in the verifier.

PART 11 · MILESTONE M7

The verifier and replanning

The verifier is **ordinary, deterministic code, not AI**. It gives the same answer every time and cannot be talked into anything. It runs four groups of checks:

Check group	Examples of what it catches
1. Physics	Car charging while it is away; battery charging and discharging at once; battery outside its limits; grid above 8 kW
2. Wishes	Room outside the comfort band while someone is home; car below its target at departure (reported as warnings to tell the person)
3. Request consistency	Every time, percentage and temperature in the request ("at 7", "90%", "26 °C") must appear in the rules; and the agents must not **invent** values nobody asked for
4. Explanation grounding	Every number and time in the final answer must come from the actual results; every warning must be told; no "all wishes met" if one was not met

Replanning

If a check fails, the orchestrator receives the error message (e.g. which agent should fix what) and must fix it, re-run the optimizer and verify again: at most **3 replans**. FINISH is refused until verification passes. If an explanation is false, the LLM gets one rewrite; if it is still wrong, the system writes the answer itself directly from the verified numbers.

> **A real example from our runs**
>
> Request: *"I'm working from home all day tomorrow, please keep the house at most 25 degrees."*
>
> The pre-cooling agent set a maximum of 25 °C, but on its own also set a **minimum of 22 °C**.
>
> The verifier rejected it: `comfort_min_c 22 is not supported by the request`.
>
> The orchestrator asked the agent again, the agent removed the invented value, and the second check passed. The person was also told honestly that the room would reach 26.9 °C at the hottest time, because the AC is too small for that heat.

Tests that inject wrong rules on purpose confirm the loop works: the mistake is caught, repaired, and the final rules equal the intended ones.

PART 12 · EVALUATION

Results

Same day, two ways

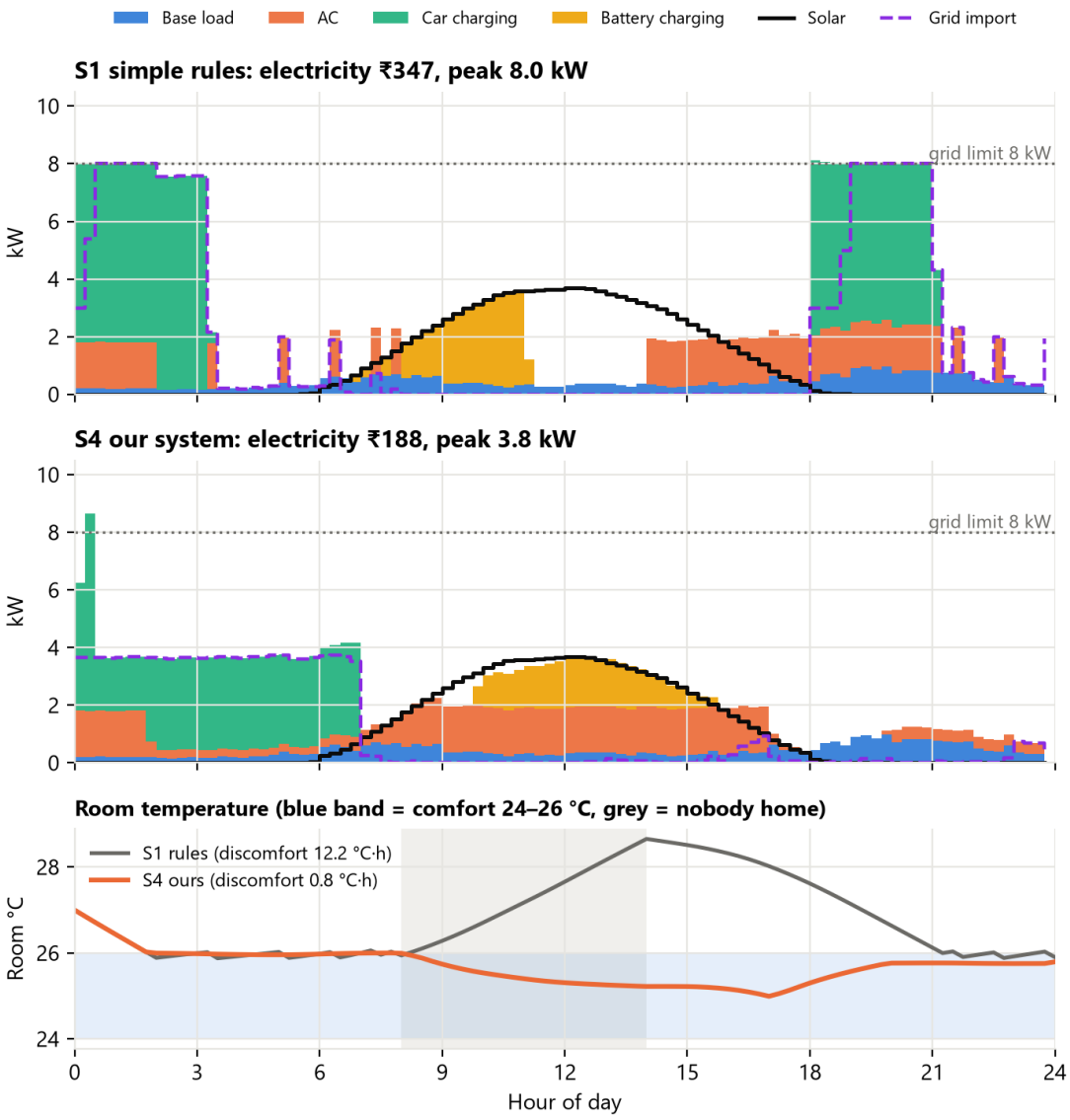


Figure 4. The example day from Part 10 (6 May 2026), both scored on the real weather. Simple rules charge the car at full power right away and push the grid to the 8 kW limit; the house heats up while empty. Our system charges the car slowly overnight, fills the battery with midday solar (yellow) and pre-cools from 14:00. Total use can exceed 8 kW for a moment because the battery supplies part of it; grid import (dashed) stays under the limit.

Four example days

Example day	S1 simple rules	S4 our system	S5 perfect knowledge	Saving vs S1
Early start + work from home (Wed 6 May)	₹347 · 12.2 °C·h	**₹188 · 0.8 °C·h**	₹181 · 0.8 °C·h	46%
Home all day in the heat (Wed 13 May)	₹312 · 5.0 °C·h	**₹200 · 4.5 °C·h**	₹192 · 4.3 °C·h	36%

Example day	S1 simple rules	S4 our system	S5 perfect knowledge	Saving vs S1
Weekend trip, car full (Sat 9 May)	₹339 · 3.2 °C·h	**₹228 · 1.1 °C·h**	₹225 · 0.0 °C·h	33%
Monsoon day, solar-only battery (Thu 30 Jul)	₹276 · 1.0 °C·h	**₹151 · 0.5 °C·h**	₹158 · 0.0 °C·h	45%

Each cell shows electricity on the real day and discomfort. On the monsoon day S4's electricity is even slightly below S5's, but S5 is still better overall once comfort is counted (total ₹167 vs ₹202).

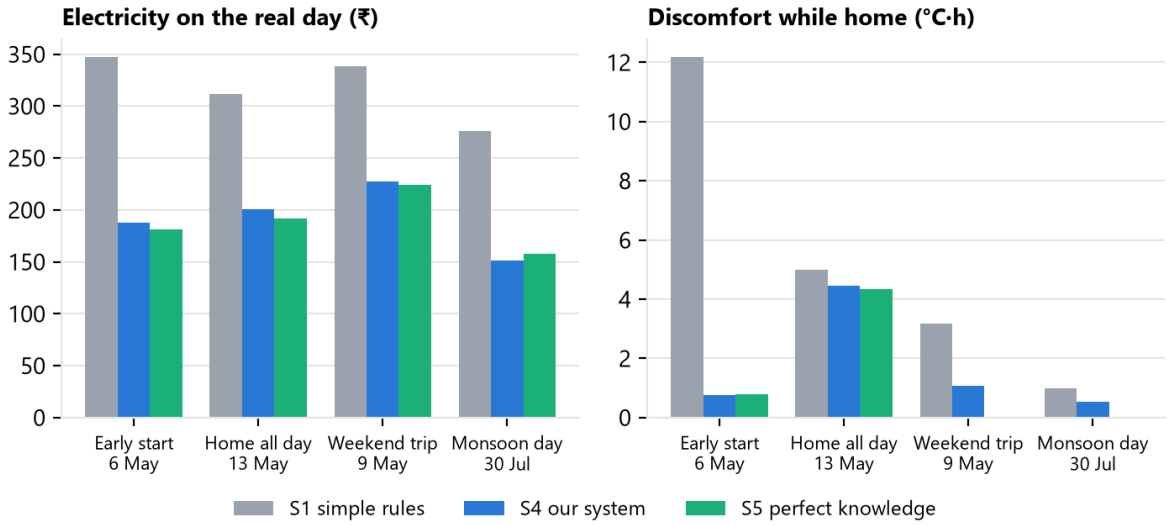


Figure 5. The four example days. Our system (blue) is close to perfect knowledge (green) and far better than simple rules (grey).

Why not let the LLM do everything? (S3)

On the first example we also asked the LLM to write the full 24-hour schedule itself, with no optimizer and no verifier:

	S4 our system	S3 pure LLM
Physical rules broken	0	**51** (car charging while away, grid over 8 kW, …)
Car short at departure	0 kWh	**20 kWh**
Discomfort	0.8 °C·h	**31 °C·h** (room up to 30.2 °C)
Electricity / total incl. penalties	₹188 / ₹247	₹85 / **₹3,633**

Its bill looks cheap only because it did not do the job. Its own reasoning even said “charge 04:00–07:00”, but the schedule it returned charged the car from 07:00 to 17:00, while the car was away. **An LLM's reasoning and its numbers can disagree.**

Key findings

Our system used **33–46% less electricity** than simple rules on all four example days and was more comfortable on every one.

It came **within about 5%** of perfect knowledge on electricity, even though it only had forecasts.

Simple rules hit the 8 kW grid limit on every example day; our system peaked at only 3–4 kW on three of the four days (6.7 kW on the weekend trip, when the car had to be full by 10:00).

The verifier caught the agents' mistakes (invented values, false explanations) and the system repaired them.

A pure LLM planner is not trustworthy for the numbers: 51 violations on one day.

Forecast quality drifts over time (Part 7).

PART 13 · MILESTONE M8

The demo

The demo is a website (built with Next.js, designed with Claude Design) that tells the story in sections: the problem, how it works, "Ask the house", "Smart vs simple" and the results. Every number on it comes from files exported by our Python code.

Running it

```
cd AIforSusProject
.venv\Scripts\activate
python -m b2g.demo.server
# then open http://localhost:3000 in the browser
```

- The four **pre-run examples** load instantly and work **without internet**.
- **Live requests:** in "Ask the house", type a request, pick a day (January–September 2026) and press "Plan my day". It needs internet and a valid API key in `.env`. It usually takes 10–60 seconds; our test request took 12 s. The result appears as a new card with the same charts and the agent steps replayed.
- Repeating the same request on the same day costs no tokens (answers are cached).
- Backup: the older Streamlit app (`streamlit run app.py`) shows the same results.

Suggested 10-minute flow

Step	Section	What to say
1	Hero	The three headline numbers: 33–46% cheaper, within 5% of perfect, 51 rules broken by a pure LLM
2	How it works	Forecast → agents → optimizer → verifier → score on real weather
3	The problem	Solar at noon, demand in the evening, monsoon dips
4	Ask the house	Pick "Home all day in the heat", press Run: the verifier rejects the invented 22 °C and the agents replan

Step	Section	What to say
5	Smart vs simple	Pick “Early start”: the four systems side by side, then the pure-LLM failure
6	Results	Forecast accuracy, the one-week table, and the honest limitations
7	Live (optional)	Type a new request live

PART 14 · HONESTY

Limitations and future work

Limitations

- Device sizes, tariff and the house's thermal model are **placeholders**, not measured from a real house.
- The house base demand is **synthetic**; the “actual” weather is ERA5 reanalysis, not a meter on our roof.
- Results come from **one week and four example days**, with **one LLM** (Qwen 3.8 27B on Groq's free tier). This is a demonstration, not a large statistical study.
- One house, not a full microgrid with many buildings.
- The plan is made once the day before; it does not re-plan during the day when the weather changes.

Future work

- Real smart-meter data and a real time-of-day tariff from a Gujarat utility.
- Evaluate over many months and both seasons, and with several LLMs.
- Rolling re-planning during the day as new forecasts arrive.
- Several houses sharing a feeder or a community battery (a real microgrid).
- A better battery ageing model and forecast uncertainty in the optimizer.

PART 15 · PRESENTATION

Questions we may be asked

Q: Why not just use the optimizer without an LLM?

The optimizer needs exact rules; people speak in sentences (“leaving half an hour earlier”, “keep it cool”). The LLM is the translator. S2 shows what the optimizer does with the default routine; the agents let a person change that routine in plain language.

Q: Why not just use the LLM?

S3 shows it: 51 rule violations, the car 20 kWh short, the room at 30 °C, on a single day.

Q: How do you know the AI did not make something up?

The verifier compares every rule with the request and every number in the explanation with the computed results; invented values are rejected and sent back for replanning.

Q: Are plans judged on the forecast?

No. Plans are made on the forecast and scored on the actual weather of that day.

Q: Is it reproducible?

Yes: fixed random seeds, LLM temperature 0, every LLM call logged, and answers cached so a re-run gives the same result for free.

Q: What is MILP?

A mathematical method that finds the best decisions over many variables while obeying all rules; here 96 time slots × several devices.

Q: Where is the B2G part?

The house moves its demand into solar hours, avoids the evening peak, stays under its grid limit and uses its own solar: what the grid wants from buildings.

Q: Why did LightGBM lose to physics for solar on the test set?

Drift: the weather model's error pattern changed in 2026. We chose the model on validation, as is correct, and report the test result honestly.

Q: Does it need a GPU?

No. Everything runs on a laptop CPU; the LLM runs in the cloud (Groq free tier) or could run locally with Ollama.

Q: What happens if the forecast is wrong?

The plan is replayed on reality: the grid covers missing solar, so the bill rises a little. That is why S4 is slightly above S5 (perfect knowledge).

PART 16 · REFERENCE

Glossary

Term	Meaning
kW / kWh	Power (speed of energy use) / energy (amount). Bills charge per kWh (“unit”).
Import / export	Taking power from the grid (you pay) / sending power to it (small credit).
Time-of-day tariff	A price that changes during the day.
Flexible load	Electricity use that can be moved in time (car, battery, pre-cooling).
Demand response	Changing when you use electricity to help the grid.
B2G (Building-to-Grid)	A building acting as a good partner of the grid.
Microgrid	A small local network with its own generation.
Pre-cooling	Cooling a house in advance so it stays cool later.
COP	Coefficient of performance: heat removed per unit of electricity (3.3 for our AC).
Slot	One 15-minute step; 96 per day.
LLM	Large language model: an AI that understands and writes text.
Agent / orchestrator	An LLM with a job and tools / the manager agent.
ReAct	The think → act → observe loop the agents use.
JSON / schema	A strict machine-readable format / the rules that format must follow.

Term	Meaning
Temperature 0	LLM setting with no randomness, so answers repeat.
Forecast / persistence	A prediction of the future / the “tomorrow = today” guess.
LightGBM	A fast machine-learning model built from many small decision trees.
nMAE	Average forecast error as a % of the average value.
Drift	Data patterns changing over time, so old models get worse.
Train / validation / test	Data to learn from / to choose settings / to score at the end only.
ERA5	A scientific reconstruction of past weather, used as “actual” weather.
1R1C model	Simple building temperature model with one resistance and one thermal mass.
MILP	Mixed-integer linear programming: finds the best decision under rules.
CBC / PuLP	The free solver / the Python library we use to call it.
Constraint / objective	A rule the plan must obey / what the plan minimises.
Settlement	Replaying a plan on the actual weather to score it.
°C·h	Degrees outside the comfort band × hours.
Self-consumption	Share of your solar you use yourself rather than export.
Baseline / oracle	A simple method to compare against / a system with perfect future knowledge (S5).
Verifier	Our checking code; replanning = sending the plan back to be fixed.

PART 17 · REFERENCE

Appendix: milestones, code map and commands

Where things are

Folder / file	What it contains
orchestrator_agent_react.py, tools.py, config.py, *_agent.md	The original paper's system (S0), kept runnable
b2g/config/default.yaml	All placeholder numbers: devices, tariff, comfort, solver weights
b2g/agents/llm_client.py	One LLM client for every call: providers, logging, cache, retries
b2g/data/	Weather download, solar and base-load profiles, the 15-minute dataset
b2g/forecasting/	Persistence, physics and LightGBM forecasters; training and scoring
b2g/models/	Battery, car, thermal and site models
b2g/solver/milp.py	The MILP optimizer (solve_day)
b2g/sim/	One day's data, the settlement on the actual weather
b2g/baselines/	S1 simple rules, S2 forecast + optimizer, S3 pure LLM
b2g/agents/	Orchestrator, rule schemas, tools, prompts for the agents

Folder / file	What it contains
b2g/verifier/	The four check groups and the request-fact reader
b2g/demo/	Pre-run examples, web data export, the live server
web/	The Next.js website (built site in web/out)
tests/	139 automated tests (all passing)
docs/B2G_PLAN.md, docs/DECISIONS.md	The plan with milestones, and every design decision with its reason

Reproducibility

- Fixed random seeds; LLM temperature 0.
- Every LLM call is logged (prompt, answer, tokens, time) in `results/runs/`.
- Answers are cached by a hash of the prompt in `results/cache/`: a re-run costs 0 tokens and gives the same result.
- API keys live only in `.env`, which is never shared.

Commands

```
pip install -r requirements.txt -r requirements-api.txt -r requirements-b2g.txt
pytest -q                                  # run all tests
python -m b2g.data.build                   # M2: build the dataset
python -m b2g.forecasting.train            # M3: train and score forecasters (~25 s)
python -m b2g.eval.compare_baselines       # M5: one week, S1 / S2 / S5
python -m b2g.cli --system S4 --date 2026-05-06 --request "..."   # M6/M7: one request
python -m b2g.cli --system S3               # pure LLM on the demo request
python -m b2g.demo.precompute              # M8: pre-run the 4 examples
python -m b2g.demo.server                  # the website + live requests on :3000
```